

VirtuEx 성능 개선 감사 보고서 (11차)

VirtuEx Performance Improvement Audit Report (11th)

- 감사일: 2026-03-24
- 감사 범위: 프론트엔드 렌더링/Memoization, 번들/코드 분할, API/네트워크 효율, 캐싱/메모리 관리, DB/서버 성능, WebSocket/실시간 처리, 리스트/가상화, 기타
- 감사 방법: 소스 코드 정적 분석, 렌더링 패턴 추적, 쿼리 패턴 추적, 번들/캐싱 전략 검토
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 발견 사항 정리만 수행하며, 수정은 포함하지 않음
- 11차 강화 기준: 컴포넌트 200줄 초과 플러그(8차: 300줄), 20건 이상 리스트 가상화 필수, 모든 비동기 작업 로딩 상태 검사, useEffect 의존성 정확성 감사, prop drilling 검사, Zustand 셀렉터 최적화 검사

이전 감사 대비 수정 현황 (10차 → 11차)

이전 이슈	상 태	비고
[PF-M-01] AssetList currentSymbols map+join 매 렌더	미 수 정	AssetList.tsx:98 — 여전히 assets.map(a => a.symbol).join(', ') 매 렌더 실행 (5차 연속)
[PF-M-02] 주문 분석 탭 trades.filter() 반복	미 수 정	orders/page.tsx:189 — breakdown 내 trades.filter() O(N*M) 패턴 유지 (3차 연속)
[PF-M-03] PortfolioHistoryChart totalValue 재계산	미 수 정	PortfolioHistoryChart.tsx:129 — useMemo 의존성에 totalValue 포함 그대로 (5차 연속)
[API-M-01] 체결 내역 프론트엔드 서버 페이징 이제이션 미활용	미 수 정	useTradeHistory() 여전히 limit/offset 미전달 (5차 연속)
[API-L-01] 리더보드 period/sortBy 서버 미지원	미 수 정	useLeaderboard.ts:49 — 여전히 limit: 100만 전달, period/sortBy 서버 미전달
[BD-M-01] 주문 페이지 894줄 단일 파일	미 수 정	orders/page.tsx — 894줄 유지 (3차 연속)
[BD-M-02] 리더보드 페이지 639줄	미 수 정	leaderboard/page.tsx — 639줄 유지 (3차 연속)
[BD-M-03] 마이페이지 847줄	미 수 정	mypage/page.tsx — 847줄 유지 (변동 없음)

[BD-M-04] 대시보드 AI 분석 핸들러 로직 잔존	미수정	dashboard/page.tsx:148-189 — 약 40줄 handleAiAnalysis 그대로. news/page.tsx:134-180에도 동일 중복
[BD-L-01] PortfolioAnalytics 904줄	미수정	PortfolioAnalytics.tsx — 904줄 유지 (3차 연속)
[CM-M-01] CandlestickChart chartType 변경 시 8개 시리즈 전체 재생성	미수정	CandlestickChart.tsx:319-404 동일 패턴 유지 (5차 연속)
[CM-M-02] 리더보드 prevRankMap 기간/정렬 변경 시 미초기화	미수정	leaderboard/page.tsx:235 동일 (4차 연속)
[CM-L-01] 리더보드 sortedLeaderboard 필터 변경 시 전체 재정렬	미수정	leaderboard/page.tsx:212-230 동일
[DB-M-01] 관리자 체결 감사 사용자명 enrichment N+1	미수정	order-proxy.controller.ts:162-179 동일 (5차 연속)
[DB-L-01] 거래 내역 limit=200 하드코딩	미수정	useTransactions(limit = 200) 동일
[WS-M-01] useWebSocket symbols 변경 시 전체 목록 재전송	미수정	useWebSocket.ts:196-198 subscribe 시 전체 목록 전송 (5차 연속)
[WS-L-01] 종목 상세 단일 심볼 배열 래핑	미수정	실질 영향 미미로 보류
[LV-M-01] 체결 내역 탭 가상화 없이 전체 렌더링	미수정	orders/page.tsx:824 — filteredTrades.map() 전체 DOM 렌더링
[LV-M-02] 리더보드 100건 가상화 없이 전체 렌더링	미수정	leaderboard/page.tsx:471 — sortedLeaderboard.map() 전체 렌더링
[ETC-M-01] AI 분석 모달 대시보드/뉴스 중복	미수정	뉴스 페이지(news/page.tsx:356-484)에서 여전히 인라인 AI 모달 약 130줄 중복 유지 (부분 수정 상태 그대로)
[ETC-M-02] useTradeHistory 마이페이지 불필요 폴링	미수정	mypage/page.tsx:139 — 여전히 10초 폴링 활성 상태
[ETC-L-01] AssetList FLIP void el.offsetHeight 강제 리플로우	미수정	AssetList.tsx:200 — 개별 요소별 리플로우

	정	
--	---	--

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 렌더링/Memoization	0	0	2	1	3
B. API/네트워크	0	0	1	1	2
C. 번들/코드 분할	0	0	4	1	5
D. 캐싱/메모리	0	0	2	1	3
E. DB/서버	0	0	1	1	2
F. WebSocket/실시간	0	0	1	0	1
G. 리스트/가상화	0	0	2	0	2
H. 기타	0	0	3	1	4
I. 폰트/리소스 로딩 (보충 감사)	0	0	1	0	1
J. 번들/중복 코드 (보충 감사)	0	0	1	1	2
합계	0	0	18	7	25

A. 렌더링/Memoization — 3건

[PF-M-01] AssetList currentSymbols — assets.map().join() 매 렌더 실행 (미수정, 5차 연속)

- **현상:** AssetList.tsx:98 에서 useMemo(() => assets.map(a => a.symbol).join(', '), [assets]) . assets 참조는 WebSocket 틱마다 새로 생성되므로 매번 100개+ 심볼을 map+join 실행. assetsVersion useMemo에서 이전 문자열과 비교하여 정렬 재실행을 방지하지만, join 자체의 O(N) 연산은 매번 발생
- **위치:** frontend/src/components/market/AssetList.tsx:98-108
- **영향:** 500ms마다 100+ 심볼의 join 및 문자열 비교 수행
- **권장:** assets.length 와 첫/마지막 심볼만 O(1) 비교하거나, 상위 컴포넌트에서 심볼 Set의 안정적 ref 전달
- **심각도:** Medium

[PF-M-02] 주문 분석 탭 breakdown — trades.filter() O(N*M) 반복 (미수정, 3차 연속)

- **현상:** orders/page.tsx:189 의 AnalysisTab 내 stats useMemo에서 symbolMap 으로 buyTotal/sellTotal을 이미 집계했음에도, breakdown 계산 시 trades.filter(tr => tr.symbol === symbol && tr.buyerId === userId) 호출로 O(N*M) 복잡도 발생
- **위치:** frontend/src/app/(main)/orders/page.tsx:187-194
- **영향:** 체결 내역 1000건 x 심볼 50종 시 50,000번 필터 반복
- **권장:** symbolMap 순회 시 buyCount/sellCount를 동시 집계하여 filter 호출 제거
- **심각도:** Medium

[PF-M-03] PortfolioHistoryChart — totalValue 변경 시 전체 시계열 재구성 (미수정, 5차 연속)

- **현상:** PortfolioHistoryChart.tsx:127-129 에서 buildHistoryFromTransactions 의 useMemo 의존성에 `totalValue` 가 포함. 포트폴리오 가격이 폴링으로 변할 때마다 전체 거래 내역을 재정렬하고 시계열 데이터를 재구성. 함수 내부에서 `sorted = [...transactions].sort(...)` (52줄), `allPoints` 루프 (65-75 줄), `periodPoints` 루프 (83-89줄), `dayMap` 루프 (100-104줄) 등 다수의 배열 연산이 반복됨
- **위치:** frontend/src/components/portfolio/PortfolioHistoryChart.tsx:127-129
- **영향:** 거래 내역 200건+ 시 포트폴리오 가치 변경마다 불필요한 배열 복사/정렬 반복
- **권장:** 거래 내역 기반 시계열은 `transactions / period` 변경 시에만 계산하고, `totalValue` 변경 시에는 마지막 포인트만 교체
- **심각도:** Low

B. API/네트워크 효율 — 2건

[API-M-01] 체결 내역 — 프론트엔드에서 서버 페이지네이션 미활용 (미수정, 5차 연속)

- **현상:** `useTradeHistory()` 혹은 `useOrders.ts:194-219` 이 `/api/orders/trades/history` 에 `limit / offset` 파라미터를 전달하지 않고 전체 체결 내역을 가져옴. 주문 페이지(`orders/page.tsx:369`)와 마이페이지(`mypage/page.tsx:139`)에서 각각 동일 호출. 주문 페이지의 `useOrders()` 는 이미 `limit / offset` 을 서버에 전달하는 API-M-01 패치가 적용(`orders/page.tsx:364-376`)되었으나, `useTradeHistory()` 는 미적용
- **위치:** frontend/src/hooks/useOrders.ts:194-219
- **영향:** 체결 내역 누적 시 응답 크기 선형 증가. 마이페이지에서는 통계 집계용이므로 전체 데이터가 필요하나, 주문 페이지 체결 탭에서는 페이지네이션 적용 필요
- **권장:** `useTradeHistory(limit?, offset?)` 파라미터 추가. 주문 체결 탭에서 페이지 단위 요청, 분석 탭에서는 서버 측 집계 API 제공
- **심각도:** Medium

[API-L-01] 리더보드 period/sortBy — 서버 미지원으로 클라이언트 전체 재정렬

- **현상:** `useLeaderboard.ts:49` 에서 `api.get('/api/portfolio/leaderboard', { params: { limit: 100 } })` 로 항상 100명만 조회. `period` 와 `sortBy` 가 `queryKey`에는 포함되지만 (`useLeaderboard.ts:45`) 서버 파라미터로 미전달 — 모든 기간/정렬 조건에서 동일 데이터를 조회하고 클라이언트에서 재정렬/필터링(`leaderboard/page.tsx:212-230`)
- **위치:** frontend/src/hooks/useLeaderboard.ts:49
- **영향:** 기간 변경 시에도 동일 데이터를 반복 요청, 서버 측 기간별 수익률 계산 미활용. 다만 TanStack Query의 `queryKey` 에 `period/sortBy`가 포함되어 있어, 기간 변경 시 캐시 미스로 불필요한 서버 요청 발생
- **권장:** 백엔드 리더보드 API에 `period` 파라미터 지원 추가. 또는 `queryKey`에서 `period/sortBy`를 제거하고 클라이언트 재정렬 시 캐시된 데이터 재사용
- **심각도:** Low

C. 번들/코드 분할 — 5건

[BD-M-01] 주문 페이지 — 894줄 단일 파일에 AnalysisTab + OrdersPage 동거 (미수정, 3차 연속)

- **현상:** `orders/page.tsx` 가 894줄로, `StatusDropdown` (44-108줄), `AnalysisTab` (111-344줄), `OrdersPage` (346-894줄) 세 컴포넌트를 단일 파일에 포함. `AnalysisTab`은 별도 컴포넌트로 추출되었으나 `dynamic import` 미적용으로 분석 탭 미사용 시에도 코드가 로드됨. 11차 강화 기준(200줄) 대폭 초과
- **위치:** frontend/src/app/(main)/orders/page.tsx:1-894
- **영향:** 주문 목록만 조회 시 분석 탭의 차트 로직(시간 분포, 심볼별 통계 등)이 불필요하게 번들에 포함
- **권장:** `AnalysisTab`을 별도 파일로 분리 후 `dynamic(() => import(...))` 으로 지연 로드. `StatusDropdown`도 공유 UI 컴포넌트로 추출

- 심각도: Medium

[BD-M-02] 리더보드 페이지 — 639줄 단일 컴포넌트 (미수정, 3차 연속)

- 현상: `leaderboard/page.tsx` 가 639줄로, 기간 필터, 정렬, FLIP 애니메이션, 팔로우/카피트레이드 모달, 사용자 프로필 모달, 언팔로우 확인 모달 등 모든 로직을 포함. 11차 강화 기준(200줄) 3배 초과. `MedalBadge` (79-93줄), `RankChangeIndicator` (97-121줄), `calcAbsolutePnl` (71-75줄) 등의 유틸 컴포넌트가 같은 파일에 동거
- 위치: `frontend/src/app/(main)/leaderboard/page.tsx:1-639`
- 영향: 초기 렌더 시 카피트레이드/프로필 모달 등 미사용 코드까지 전부 로드
- 권장: 리더보드 테이블 행을 `LeaderboardRow` 컴포넌트로 분리, 카피트레이드/프로필 모달을 `dynamic import`로 전환, `MedalBadge` / `RankChangeIndicator` 를 별도 파일로 추출
- 심각도: Medium

[BD-M-03] 마이페이지 — 847줄 단일 컴포넌트 (미수정, 변동 없음)

- 현상: `mypage/page.tsx` 가 847줄로 변동 없음. 프로필 조회, 비밀번호 변경 모달(자체 포커스 트랩 구현 204-220줄), 알림 설정 토글, 계정 초기화 2단계 확인, 거래 통계 집계 등을 포함. 비밀번호 변경 모달(642-806줄, 약 165줄)과 계정 초기화 로직(571-638줄, 약 68줄)이 페이지 로드 시 항상 번들에 포함. `InfoRow` (26-33줄), `RoleBadge` (36-56줄), `NotifToggle` (65-123줄) 세 개의 하위 컴포넌트도 같은 파일에 존재
- 위치: `frontend/src/app/(main)/mypage/page.tsx:1-847`
- 영향: 페이지 진입 시 비밀번호 변경/계정 초기화 코드가 불필요하게 로드. 세 개의 하위 컴포넌트가 파일 분리 없이 인라인 정의
- 권장: 비밀번호 변경 모달, 계정 초기화 UI를 각각 별도 컴포넌트로 분리 후 `dynamic import`. `useFocusTrap` 혹은 이미 존재(`hooks/useFocusTrap.ts`)하므로 자체 포커스 트랩 대신 공유 훅 활용
- 심각도: Medium

[BD-M-04] 대시보드/뉴스 AI 분석 핸들러 — `handleAiAnalysis` 로직 2곳 중복 (미수정)

- 현상: `dashboard/page.tsx:148-189` 에서 `handleAiAnalysis` 콜백(약 40줄)이 여전히 페이지 컴포넌트에 위치. `news/page.tsx:134-180` 에도 거의 동일한 `handleAiAnalysis` 로직이 중복 존재(약 46줄). 두 함수의 핵심 로직은 동일하며(뉴스 조회 → 필터 → AI 요약 API 호출 → 결과 설정), 유일한 차이는 뉴스 페이지에서 `activeTab` 카테고리 동적으로 전달하는 것(`news/page.tsx:143`)과 대시보드에서 'CRYPTO' 고정 전달(`dashboard/page.tsx:157`)
- 위치: `frontend/src/app/(main)/dashboard/page.tsx:148-189` , `frontend/src/app/(main)/news/page.tsx:134-180`
- 영향: AI 분석 로직 변경 시 두 파일을 각각 수정해야 함. 약 86줄의 중복 코드
- 권장: `useAiNewsAnalysis(category)` 커스텀 훅으로 추출하여 대시보드/뉴스 페이지 모두에서 재사용
- 심각도: Medium

[BD-L-01] PortfolioAnalytics — 904줄 단일 컴포넌트 (미수정, 3차 연속)

- 현상: `PortfolioAnalytics.tsx` (904줄)이 `recharts`의 `PieChart`, `LineChart`, `BarChart` 등을 직접 `import`. 이 컴포넌트 자체는 `portfolio/page.tsx` 에서 `dynamic(() => import(...))` 으로 코드 스플리팅되어 있으므로 초기 번들에는 미포함. 그러나 904줄 단일 컴포넌트로 11차 기준(200줄) 4.5배 초과
- 위치: `frontend/src/components/portfolio/PortfolioAnalytics.tsx:1-904`
- 영향: `analytics` 탭 로드 시 904줄 전체가 한 번에 파싱. 각 분석 섹션(자산 배분, 일별 손익, 위험 지표 등)이 항상 로드됨
- 권장: 각 분석 섹션(`PieChart`, `DailyPnL`, `RiskMetrics` 등)을 별도 컴포넌트로 분리하여 모듈화
- 심각도: Low

D. 캐싱/메모리 관리 — 3건

[CM-M-01] CandlestickChart — `chartType` 변경 시 8개 시리즈 전체 재생성 (미수정, 5차 연속)

- **현상:** CandlestickChart.tsx:319-404 에서 chartType 변경 시 기존 시리즈 전부 제거 후 재생성. 캔들 ↔ 라인 전환 시 메인 시리즈뿐 아니라 볼륨, SMA 3개,布林저 밴드 3개까지 총 8개 시리즈를 destroy/recreate. chart.removeSeries() 8회 → chart.addXxxSeries() 8회의 순차 호출이 단일 useEffect 내에서 실행
- **위치:** frontend/src/components/chart/CandlestickChart.tsx:319-404
- **영향:** 차트 타입 전환 시 눈에 띄는 깜빡임과 지연 (시리즈 생성 → 데이터 세팅의 2단계 렌더)
- **권장:** 메인 시리즈만 교체하고 보조 지표 시리즈는 유지, 또는 두 시리즈를 모두 생성 후 visible 토글로 전환
- **심각도:** Medium

[CM-M-02] 리더보드 prevRankMap — 기간/정렬 변경 시 미초기화 (미수정, 4차 연속)

- **현상:** leaderboard/page.tsx:235 의 prevRankMap.current 가 컴포넌트 생애 동안 이전 순위 데이터를 계속 유지. period 나 sortMode , investedOnly , copyTradeOnly 변경 시에도 초기화되지 않아 잘못된 순위 변동 애니메이션 발생. useEffect 의존성([sortedLeaderboard] , 286줄)에 period / sortMode 등이 포함되지 않아 필터 변경 시 이전 순위 맵이 그대로 사용됨
- **위치:** frontend/src/app/(main)/leaderboard/page.tsx:235
- **영향:** 필터 변경 후 순위 변동 인디케이터가 실제와 다르게 표시. FLIP 애니메이션이 의미 없는 이동을 수행
- **권장:** period , sortMode , investedOnly , copyTradeOnly 변경 시 prevRankMap.current = new Map() 호출
- **심각도:** Medium

[CM-L-01] 리더보드 sortedLeaderboard — 필터 변경 시 전체 재정렬 (미수정)

- **현상:** leaderboard/page.tsx:212-230 의 sortedLeaderboard useMemo가 leaderboard , sortMode , investedOnly , copyTradeOnly , copyTradingUserIds 5개 의존성을 가짐. 토글 변경마다 전체 배열을 filter → sort → map으로 재계산
- **위치:** frontend/src/app/(main)/leaderboard/page.tsx:212-230
- **영향:** 사용자 100명+ 시 불필요한 정렬 반복. 단, 현재 limit: 100 으로 100명 미만이므로 실질적 영향은 제한적
- **권장:** 정렬 결과를 캐싱하고 필터만 별도 useMemo로 분리
- **심각도:** Low

E. DB/서버 성능 — 2건

[DB-M-01] 관리자 체결 감사 — 사용자명 enrichment N+1 패턴 (미수정, 5차 연속)

- **현상:** order-proxy.controller.ts:162-179 에서 체결 내역 조회 후 사용자명을 enrichment하기 위해 POST /users/by-ids 호출. 현재는 한번의 배치 호출이지만, 체결 내역의 userId가 다수(100+)일 경우 대량 ID 배열 전송. userMap 결과가 캐싱되지 않아 동일 사용자에 대해 매 요청마다 반복 조회
- **위치:** backend/services/api-gateway/src/proxy/order-proxy.controller.ts:162-179
- **영향:** 관리자 페이지 체결 내역 조회 시 추가 네트워크 홉, userId 100개+ 시 요청 크기 증가
- **권장:** Redis 캐싱으로 userId → username 매핑 캐시 (TTL 5분), 또는 order-engine DB에 사용자명 비정규화 저장
- **심각도:** Medium

[DB-L-01] 거래 내역(transactions) — limit=200 하드코딩 (미수정)

- **현상:** useTransactions(limit = 200) 혹에서 기본 200건을 가져옴. PortfolioHistoryChart 에서 사용하지만, 기간 필터('1W' 등)가 적용되어도 항상 200건을 전부 가져온 후 클라이언트에서 기간 필터링 수행 (PortfolioHistoryChart.tsx:83-89)
- **위치:** frontend/src/hooks/usePortfolio.ts:240
- **영향:** 1주 조회 시에도 불필요한 90일+ 거래 데이터 전송
- **권장:** 기간에 따라 limit 또는 since 파라미터를 동적으로 조정
- **심각도:** Low

F. WebSocket/실시간 처리 — 1건

[WS-M-01] useWebSocket — symbols 배열 변경 시 전체 목록 재전송 (미수정, 5차 연속)

- 현상: useWebSocket.ts:196-198 에서 symbols 변경 시 socket.emit('subscribe', { symbols }) 로 전체 심볼 목록을 재전송. unsubscribe는 diff 기반(removed 계산 후 개별 unsubscribe, 188-192줄)이지만, subscribe는 항상 전체 목록을 전송
 - 위치: frontend/src/hooks/useWebSocket.ts:194-198
 - 영향: 심볼 100개+ 시 관심종목 1개 추가/제거 시에도 전체 100개 심볼 재전송
 - 권장: subscribe도 diff 기반으로 신규 추가된 심볼만 전송: `const added = symbols.filter(s => !prevSymbolsRef.current.includes(s))` 후 added만 전송
 - 심각도: Medium
-

G. 리스트/가상화 — 2건

[LV-M-01] 체결 내역 탭 — 전체 filteredTrades를 가상화 없이 렌더링 (미수정)

- 현상: orders/page.tsx:824 에서 filteredTrades.map() 으로 전체 체결 내역을 DOM에 직접 렌더링. useTradeHistory() 가 limit 없이 전체 데이터를 가져오므로, 체결 내역 100건 이상 시 DOM 노드가 비례 증가. 페이지네이션 미적용, 가상화(windowing) 미적용
- 위치: frontend/src/app/(main)/orders/page.tsx:824
- 영향: 체결 내역 500건+ 시 DOM 과다 생성으로 스크롤 성능 저하 및 메모리 증가
- 권장: (1) 클라이언트 측 페이지네이션 추가 (Pagination 컴포넌트 활용 — 주문 탭에서 이미 사용 중) 또는 (2) react-virtual/tanstack-virtual로 리스트 가상화
- 심각도: Medium

[LV-M-02] 리더보드 — sortedLeaderboard 최대 100건 가상화 없이 전체 렌더링 (미수정)

- 현상: leaderboard/page.tsx:471 에서 sortedLeaderboard.map() 으로 최대 100명의 사용자를 가상화 없이 전체 렌더링. 각 행에 팔로우/카피 트레이드 버튼, 프로필 모달 트리거, FLIP 애니메이션 ref 등 복잡한 렌더 로직 포함. 11차 강화 기준(20건 이상 가상화 필수) 해당
 - 위치: frontend/src/app/(main)/leaderboard/page.tsx:469-616
 - 영향: 100명 렌더 시 각 행의 ref 콜백 + 버튼 이벤트 핸들러가 100개씩 생성. FLIP 애니메이션과의 충돌 가능성
 - 권장: FLIP 애니메이션과 호환되는 가상화 라이브러리 적용, 또는 "더 보기" 버튼으로 초기 표시 20명 → 점진적 확장
 - 심각도: Medium
-

H. 기타 — 4건

[ETC-M-01] 뉴스 AI 분석 모달 — 뉴스 페이지에서 인라인 구현 유지 (부분 수정 상태 유지)

- 현상: 대시보드에서는 DashboardAiModal (162줄)로 공유 컴포넌트 추출이 완료되었으나, 뉴스 페이지 (news/page.tsx:356-484)에서는 여전히 약 130줄의 인라인 AI 분석 모달을 구현. 두 모달의 UI 구조는 동일하며 뉴스 페이지에만 marketOutlook , riskFactors 추가 섹션이 존재. handleAiAnalysis 콜백도 dashboard/page.tsx:148-189 와 news/page.tsx:134-180 에서 거의 동일한 패턴으로 중복 구현
- 위치: frontend/src/app/(main)/news/page.tsx:356-484 , frontend/src/app/(main)/dashboard/page.tsx:148-189
- 영향: 동일 버그를 두 곳에서 각각 수정해야 하며, 번들 크기 낭비 (약 130줄 중복)
- 권장: (1) DashboardAiModal 을 확장하여 marketOutlook , riskFactors 를 선택적으로 표시하는 <AiNewsAnalysisModal /> 공유 컴포넌트로 통합 (2) handleAiAnalysis 로직을 useAiNewsAnalysis() 커스텀 훅으로 추출
- 심각도: Medium

[ETC-M-02] useTradeHistory 호출 중복 — 마이페이지 통계용 10초 폴링 불필요 (미수정)

- **현상:** useTradeHistory() 가 orders/page.tsx:369 과 mypage/page.tsx:139 에서 각각 호출됨. 동일한 queryKey ['trades', 'history'] 를 사용하므로 TanStack Query의 dedup 메커니즘으로 단일 요청으로 병합됨. 그러나 두 페이지가 동시 활성화되지 않더라도 각각 10초 폴링(refetchInterval: 10000 , useOrders.ts:214)이 독립 실행. 마이페이지에서는 통계 집계(tradingStats useMemo, 155-181줄)와 최근 5건 표시(trades.slice(0, 5) , 479줄)만 필요하므로 10초 폴링 불필요
- **위치:** frontend/src/app/(main)/mypage/page.tsx:139 , frontend/src/hooks/useOrders.ts:214
- **영향:** 마이페이지 체류 시 불필요한 10초 폴링으로 서버 부하
- **권장:** 마이페이지에서는 refetchInterval 을 비활성화하거나, staleTime을 길게 설정하여 캐시된 데이터 활용
- **심각도:** Medium

[ETC-M-03] news/page.tsx handleAiAnalysis — queryClient가 의존성에 포함되나 미사용 (신규)

- **현상:** news/page.tsx:180 의 handleAiAnalysis useCallback 의존성 배열에 queryClient 가 포함되어 있으나([activeTab, aiLoading, locale, queryClient]), 함수 본문에서 queryClient 를 사용하지 않음. queryClient 는 handleRefresh (130-132줄)에서만 사용됨. 불필요한 의존성으로 인해 queryClient 참조 변경 시 handleAiAnalysis 콜백이 불필요하게 재생성됨
- **위치:** frontend/src/app/(main)/news/page.tsx:180
- **영향:** 실질적 영향은 미미하나, 의존성 배열 정확성 원칙 위반. queryClient 참조는 일반적으로 안정적이므로 빈도는 낮음
- **권장:** queryClient 를 handleAiAnalysis 의 의존성 배열에서 제거
- **심각도:** Medium

[ETC-L-01] AssetList FLIP 애니메이션 — void el.offsetHeight 강제 리플로우 (미수정)

- **현상:** AssetList.tsx:200 에서 void el.offsetHeight 로 강제 리플로우를 발생시켜 FLIP 애니메이션을 구현. 리더보드(leaderboard/page.tsx:274)에서는 void movedEls[0].offsetHeight 로 단일 리플로우만 발생시키는 최적화가 적용되었으나, AssetList에서는 paged.forEach 루프 내에서 개별 요소마다 리플로우 발생 가능
- **위치:** frontend/src/components/market/AssetList.tsx:199-200
- **영향:** 순위 변동이 많을 때 다수의 강제 리플로우로 프레임 드롭 발생 가능. 단, 3초 스로틀로 빈도는 제한됨
- **권장:** 리더보드와 동일하게 moved 엘리먼트를 수집한 후 단일 리플로우 → requestAnimationFrame으로 배치 처리
- **심각도:** Low

I. 폰트/리소스 로딩 — 1건 (보충 감사 신규)

[FONT-M-01] CDN 폰트 스타일시트 — <link rel="preload"> 후 실제 <link rel="stylesheet"> 미적용 (보충 감사 신규)

- **현상:** layout.tsx:42 에서 <link rel="preload" href="...pretendardvariable...css" as="style" /> 로 Pretendard Variable 폰트 스타일시트를 프리로드 선언하고 있으나, 이후 실제 <link rel="stylesheet" href="..."> 태그가 HTML에 없음. globals.css:61-63 에서 font-family: 'Pretendard Variable', ... 로 폰트를 참조하지만, 폰트 CSS를 로드하는 경로는 preconnect (DNS/TLS 사전 연결)에만 의존. preload 는 리소스를 미리 다운로드하지만, 스타일시트로 적용하려면 <link rel="stylesheet"> 도 필요함. 현재 동작하는 이유는 globals.css 내부에서 폰트가 시스템 폴백으로 동작하거나, 별도 경로로 로드되고 있을 가능성
- **위치:** frontend/src/app/layout.tsx:40-42 , frontend/src/app/globals.css:61-63
- **영향:** 폰트 프리로드가 의도대로 동작하지 않을 수 있으며, 브라우저가 프리로드된 리소스를 실제로 적용하지 않으면 "The resource was preloaded but not used within a few seconds" 콘솔 경고 발생 가능. FOIT(Flash of Invisible Text) 또는 FOUT(Flash of Unstyled Text) 발생 가능

- 권장: `preload` 태그 뒤에 `<link rel="stylesheet" href="...pretendardvariable...css" />` 태그를 추가하여 실제 적용을 보장. 또는 `@font-face` 를 `globals.css` 에 직접 정의하여 `next/font` 최적화 활용
 - 심각도: Medium
-

J. 번들/중복 코드 — 2건 (보충 감사 신규)

[BD-M-05] MainContent `pb-20 md:pb-0` — `md(768px)~lg(1024px)` 구간 BottomNav 겹침 (보충 감사 신규)

- 현상: MainContent.tsx:24 에서 `<main>` 태그에 `pb-20 md:pb-0` 적용. BottomNav는 `lg:hidden` (1024px)에서 숨겨짐. 768px~1024px 구간에서 `md:pb-0` 이 적용되어 하단 패딩이 0이 되지만, BottomNav(56px 고정)는 여전히 표시 중. 이 구간에서 페이지 마지막 콘텐츠가 BottomNav 뒤에 가려질 수 있음
- 위치: `frontend/src/components/layout/MainContent.tsx:24` , `frontend/src/components/layout/BottomNav.tsx:23`
- 영향: 태블릿 크기(768px~1024px)에서 스크롤 최하단 콘텐츠가 BottomNav에 의해 56px 가려짐
- 권장: `pb-20 md:pb-0` 를 `pb-20 lg:pb-0` 으로 변경하여 BottomNav hidden 브레이크포인트와 일치시킴
- 심각도: Medium

[BD-L-02] Prisma 쿼리 로깅 — 환경별 분기 미설정 (보충 감사 신규)

- 현상: 5개 마이크로서비스(`user-auth` , `portfolio` , `order-engine` , `market-data` , `chat`)의 PrismaService 가 extends PrismaClient 로만 선언되어 있고, `super()` 호출 시 `log` 옵션을 전달하지 않음. Prisma 기본값은 로깅 비활성(`log: []`). 개발 환경에서도 쿼리 로깅이 비활성화되어 디버깅 시 SQL 추적 불가
 - 위치: `backend/services/user-auth/src/infrastructure/persistence/prisma/prisma.service.ts:12` , 동일 패턴 5개 서비스
 - 영향: 프로덕션에서 쿼리 로깅이 비활성화된 것은 정상이나, 개발 환경에서도 동일하여 느린 쿼리 디버깅이 어려움. N+1 패턴(DB-M-01)과 같은 성능 이슈 발견이 지연
 - 권장: `NODE_ENV === 'development'` 일 때 `super({ log: ['query', 'warn', 'error'] })` 적용하여 개발 시 쿼리 추적 활성화
 - 심각도: Low
-

종합 평가

11차 성능 감사(보충 감사 포함)에서 총 25건의 이슈를 발견하였습니다. 10차(21건) 대비 4건 증가로, 본 감사 신규 1건([ETC-M-03])과 보충 감사 신규 3건이 추가되었고 기존 이슈의 수정은 없었습니다.

10차 대비 변동 사항

- 수정된 이슈: 0건
- 신규 이슈 (본 감사): 1건 — [ETC-M-03] `news/page.tsx` `handleAiAnalysis` `useCallback` 의존성 배열에 미사용 `queryClient` 포함
- 신규 이슈 (보충 감사): 3건 — [FONT-M-01] CDN 폰트 프리로드 후 `stylesheet` 미적용, [BD-M-05] MainContent `md~lg` 구간 BottomNav 겹침, [BD-L-02] Prisma 쿼리 로깅 환경별 분기 미설정
- 증가/약화된 이슈: 0건 (줄 수 변동 없음)
- 연속 미수정 5차 이상 이슈: PF-M-01(5차), PF-M-03(5차), API-M-01(5차), CM-M-01(5차), DB-M-01(5차), WS-M-01(5차) — 총 6건

잔존 이슈 요약

가장 시급한 개선 항목은:

1. 주문 페이지 **894**줄 단일 파일 (BD-M-01, 3차 연속) — AnalysisTab을 별도 파일 + dynamic import로 분리
2. 마이페이지 **847**줄 단일 컴포넌트 (BD-M-03) — 비밀번호 변경/계정 초기화 모달 분리
3. 체결 내역 서버 페이지네이션 미활용 (API-M-01, 5차 연속) — useTradeHistory에 limit/offset 추가
4. 뉴스 **AI** 모달 중복 (ETC-M-01) — 대시보드는 분리 완료, 뉴스 페이지도 공유 컴포넌트로 전환 필요
5. **5차 연속** 미수정 이슈 **6건** — 장기 미해결 기술 부채로 누적 중

Zustand 스토어 셀렉터 최적화 검사 결과

모든 Zustand 스토어 사용처를 검사한 결과, 셀렉터 패턴이 적절히 적용되어 있습니다:

- `useAuthStore((s) => s.isAuthenticated)` — 필요한 필드만 선택 (불필요한 리렌더 방지)
- `useSettingsStore((s) => s.locale)` — 단일 필드 셀렉터
- `useSettingsStore((s) => s.notificationPrefs[prefKey])` — 세밀한 셀렉터로 특정 알림 설정만 구독

useEffect 의존성 정확성 검사 결과

대시보드, 주문, 마이페이지, 리더보드, 뉴스 페이지의 모든 useEffect를 검사한 결과:

- 대부분의 useEffect가 올바른 의존성을 가지고 있음
- `leaderboard/page.tsx:250-286` 의 FLIP 애니메이션 useEffect — `sortedLeaderboard` 만 의존성에 포함되어 있으나, `period / sortMode` 변경 시 `prevRankMap` 초기화가 누락됨 (CM-M-02로 이미 보고)
- `news/page.tsx:180` 의 `handleAiAnalysis` — 미사용 `queryClient` 가 의존성에 포함 (ETC-M-03으로 신규 보고)

보충 감사 — 검증 완료 항목 (이슈 아님)

CandlestickChart 이벤트 리스너 (정상):

- 메인 차트: `window.resize` 리스너 1개 (생성/해제 정상 쌍), `debounce 100ms` 적용
- RSI 차트: `window.resize` 리스너 1개 + `timeScale.subscribeVisibleLogicalRangeChange` 2개 (양방향 동기화). 모두 `cleanup` 함수에서 `unsubscribe` + `removeEventListener` 정상 해제
- 전체: 최대 5개 이벤트 리스너 (`resize 2` + `range sync 2` + `debounce timer 1`). 적절한 수준

CSS-in-JS 런타임 비용 (없음):

- `styled-components`, `emotion` 등 CSS-in-JS 라이브러리 미사용. 전체 스타일링은 Tailwind CSS 유틸리티 클래스 기반

next/dynamic 활용 (양호):

- `CandlestickChart`, `RichEditor`, `PortfolioAnalytics`, `NotificationBell`, 커뮤니티 탭 4개 — 주요 heavy 컴포넌트에 `dynamic(() => import(...), { ssr: false })` 적용 확인

이미지 최적화 (해당 없음):

- 프로젝트에서 `<img>` 태그 또는 정적 이미지 파일 사용이 거의 없음. 아이콘은 Lucide React SVG, 로고는 SVG 컴포넌트 (`VirtuExLogo`). 사용자 업로드 이미지는 `community RichEditor`를 통해 서버 저장 후 URL 참조.
`next/image` 최적화 대상 이미지 없음

전체적으로 10차에서 High 이상 이슈 0건을 달성한 이후 현상 유지 중이며, Medium 이슈 18건 중 3차 이상 연속 미수정 건이 증가하고 있어 기술 부채의 체계적 해소가 필요합니다.